

Lab 4 - Part 2: Document Classification, Sentiment Analysis & Topic Modeling

Course: Natural Language Processing

Objectives:

- Build document classifiers (intro + advanced)
- Perform sentiment analysis on different domains
- Discover topics using unsupervised learning
- Compare different feature extraction methods

Instructions

1. Complete all exercises marked with `# YOUR CODE HERE`
2. **Answer all written questions** in the designated markdown cells
3. Save your completed notebook
4. **Push to your Git repository and send the link to:** yoroba93@gmail.com

Personal Analysis Required

This lab contains questions requiring YOUR personal interpretation.

Use Cases Covered

Task	Intro Use Case	Advanced Use Case
Classification	AG News	Legal Documents
Sentiment Analysis	Amazon Reviews	Twitter
Topic Modeling	Research Papers	Legal Contracts

Setup

```
# Install required libraries (uncomment if needed)
!pip install datasets scikit-learn nltk pandas numpy matplotlib seaborn wordcloud gensim
```

```
satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.2)
satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.7.0)
satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0-)
satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.18.0)
satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.2)
satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.1)
satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.6.1)
satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0-)
satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.5.1)
satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.28.1)
satisfied: typer<0.26.0,>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.21.1)
satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (4.12.0)
```

```

satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=
satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=20
satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=20
satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (4
satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->data
satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0
satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.
satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.24.0->
satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=
satisfied: markdown-it-py==2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->hug
satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->
satisfied: mdurl==0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py==2.2.0->rich>=13.8.0->typer<0.26.0,
4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
27.9/27.9 MB 21.5 MB/s eta 0:00:00

d packages: gensim
.d gensim-4.4.0

```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import re
import warnings
warnings.filterwarnings('ignore')

# NLTK
import nltk
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Scikit-learn
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score
from sklearn.decomposition import LatentDirichletAllocation, NMF
from sklearn.pipeline import Pipeline

# Hugging Face datasets
from datasets import load_dataset

print("Setup complete!")
```

Setup complete!

```
# Common preprocessing function
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_simple(text):
    """Basic preprocessing: lowercase, remove punctuation."""
    text = str(text).lower()
    text = re.sub(r'^a-zA-Z\s]', '', text)
    return ' '.join(text.split())

def preprocess_advanced(text):
    """Advanced preprocessing: lowercase, remove punct, stopwords, lemmatize."""
    text = str(text).lower()
    text = re.sub(r'^a-zA-Z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 2]
    return ' '.join(tokens)

print("Preprocessing functions ready!")
```

```
Preprocessing functions ready!
```

- ✓ PART A: Document Classification

We will work with two use cases:

1. **Intro:** News Topic Classification (AG News)
2. **Advanced:** Legal Document Classification (LexGLUE)

✓ A.1 Intro: News Topic Classification (AG News)

Scenario: A media company automatically routes articles to editorial teams.

Feature Extraction: TF-IDF

```
# Load AG News dataset
print("Loading AG News dataset...")
ag_news = load_dataset("fancyzhx/ag_news")

# Use subset for faster processing
ag_train = pd.DataFrame(ag_news['train']).sample(n=8000, random_state=42)
ag_test = pd.DataFrame(ag_news['test']).sample(n=2000, random_state=42)

# Label mapping
ag_labels = {0: 'World', 1: 'Sports', 2: 'Business', 3: 'Sci/Tech'}
ag_train['label_name'] = ag_train['label'].map(ag_labels)
ag_test['label_name'] = ag_test['label'].map(ag_labels)

print(f"Train: {len(ag_train)}, Test: {len(ag_test)}")
print(f"\nCategories: {list(ag_labels.values())}")
print(ag_train['label_name'].value_counts())
```

```
Loading AG News dataset...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
README.md: 100%                               8.07k/8.07k [00:00<00:00, 621kB/s]

data/train-00000-of-00001.parquet: 100%          18.6M/18.6M [00:00<00:00, 34.6MB/s]

data/test-00000-of-00001.parquet: 100%           1.23M/1.23M [00:00<00:00, 2.26MB/s]

Generating train split: 100%                    120000/120000 [00:00<00:00, 339970.96 examples/s]

Generating test split: 100%                     7600/7600 [00:00<00:00, 173077.44 examples/s]

Train: 8000, Test: 2000

Categories: ['World', 'Sports', 'Business', 'Sci/Tech']
label_name
Sports      2074
Sci/Tech    2021
Business    1959
World       1946
Name: count, dtype: int64
```

```
# Preprocess
ag_train['text_clean'] = ag_train['text'].apply(preprocess_simple)
ag_test['text_clean'] = ag_test['text'].apply(preprocess_simple)

# TF-IDF Vectorization
tfidf_ag = TfidfVectorizer(min_df=5, max_df=0.8, stop_words="english")
X_train_ag = X_train_ag = tfidf_ag.fit_transform(ag_train["text_clean"])
X_test_ag = X_test_ag = tfidf_ag.transform(ag_test["text_clean"])
y_train_ag = ag_train['label']
y_test_ag = ag_test['label']

print(f"TF-IDF features: {X_train_ag.shape[1]}")
```

TF-IDF features: 5766

✓ Exercise A.1: Train a News Classifier

```
# TODO: Train a Logistic Regression classifier on AG News
# 1. Create the classifier
# 2. Train it
# 3. Make predictions
# 4. Calculate accuracy and F1-score (macro)

# YOUR CODE HERE
clf_ag = LogisticRegression(max_iter=1000, random_state=42) # Create LogisticRegression clf_ag =

# Train
clf_ag.fit(X_train_ag, y_train_ag)

# Predict
```

```

y_pred_ag = clf_ag.predict(X_test_ag)

# Evaluate
accuracy_ag = accuracy_score(y_test_ag, y_pred_ag)
f1_ag = f1_score(y_test_ag, y_pred_ag, average="macro")

print(f"AG News Classification Results:")
print(f"  Accuracy: {accuracy_ag:.4f}")
print(f"  F1 (macro): {f1_ag:.4f}")

```

```

AG News Classification Results:
  Accuracy: 0.8820
  F1 (macro): 0.8812

```

```

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test_ag, y_pred_ag, target_names=list(ag_labels.values())))

```

```

Classification Report:

```

	precision	recall	f1-score	support
World	0.89	0.87	0.88	493
Sports	0.93	0.96	0.95	504
Business	0.85	0.82	0.83	474
Sci/Tech	0.85	0.87	0.86	529
accuracy			0.88	2000
macro avg	0.88	0.88	0.88	2000
weighted avg	0.88	0.88	0.88	2000

✓ A.2 Advanced: Legal Document Classification (LexGLUE - ECtHR)

Scenario: A law firm classifies court decisions by violated articles.

Feature Extraction: Bag of Words with N-grams

Challenge: Legal text is longer and uses specialized vocabulary.

```

# Load LexGLUE ECtHR dataset (European Court of Human Rights)
print("Loading LexGLUE ECtHR dataset...")
lex_glue = load_dataset("coastalcph/lex_glue", "ecthr_a", trust_remote_code=True)

# Convert to DataFrame
lex_train = pd.DataFrame(lex_glue['train'])
lex_test = pd.DataFrame(lex_glue['test'])

# Use subset (legal docs are long)
lex_train = lex_train.sample(n=min(1500, len(lex_train)), random_state=42)
lex_test = lex_test.sample(n=min(500, len(lex_test)), random_state=42)

print(f"Train: {len(lex_train)}, Test: {len(lex_test)}")
print(f"\nColumns: {lex_train.columns.tolist()}")

```

`trust\_remote\_code` is not supported anymore.
Please check that the Hugging Face dataset 'coastalcph/lex_glue' isn't based on a loading script and remove `trust\_remote\_code`.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format.
ERROR:datasets.load:`trust\_remote\_code` is not supported anymore.
Please check that the Hugging Face dataset 'coastalcph/lex_glue' isn't based on a loading script and remove `trust\_remote\_code`.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format.
Loading LexGLUE ECtHR dataset...

```

README.md: 100%                               34.1k/34.1k [00:00<00:00, 786kB/s]

ecthr_a/train-00000-of-00001.parquet: 100%      42.4M/42.4M [00:01<00:00, 34.9MB/s]

ecthr_a/test-00000-of-00001.parquet: 100%       5.68M/5.68M [00:00<00:00, 7.85MB/s]

ecthr_a/validation-00000-of-00001.parquet(...): 100%  5.26M/5.26M [00:00<00:00, 15.6MB/s]

Generating train split: 100%                    9000/9000 [00:01<00:00, 8793.41 examples/s]

Generating test split: 100%                     1000/1000 [00:00<00:00, 5364.60 examples/s]

Generating validation split: 100%                1000/1000 [00:00<00:00, 2571.61 examples/s]

Train: 1500, Test: 500

Columns: ['text', 'labels']

```

```

# Examine the data structure
print("Sample legal document (first 500 chars):")
sample_text = ' '.join(lex_train.iloc[0]['text'][:3]) # text is a list of paragraphs
print(sample_text[:500])

```

```
print(f"\nLabels (violated articles): {lex_train.iloc[0]['labels']}")
```

Sample legal document (first 500 chars):

5. The applicant, Mr Laszlo Kilyen, was born in 1972 and lives in Murgești. 6. On 10 May 2003 police officers T.M. and L.C.

Labels (violated articles): [4]

```
# Prepare data: combine text paragraphs and use first label for simplicity
def prepare_legal_text(row):
    """Join text paragraphs and truncate."""
    full_text = ' '.join(row['text'])
    return full_text[:5000] # Truncate long documents

lex_train['full_text'] = lex_train.apply(prepare_legal_text, axis=1)
lex_test['full_text'] = lex_test.apply(prepare_legal_text, axis=1)

# Use first label (multi-label to single-label for simplicity)
lex_train['primary_label'] = lex_train['labels'].apply(lambda x: x[0] if x else -1)
lex_test['primary_label'] = lex_test['labels'].apply(lambda x: x[0] if x else -1)

# Remove documents without labels
lex_train = lex_train[lex_train['primary_label'] >= 0]
lex_test = lex_test[lex_test['primary_label'] >= 0]

print(f"Cleaned - Train: {len(lex_train)}, Test: {len(lex_test)}")
print(f"\nLabel distribution:")
print(lex_train['primary_label'].value_counts().head(10))
```

Cleaned - Train: 1340, Test: 428

```
Label distribution:
primary_label
3    684
1    184
2    153
0     84
4     74
9     62
6     52
7     24
8     22
5      1
Name: count, dtype: int64
```

✓ Exercise A.2: Build a Legal Document Classifier

```
import nltk

# Download all assets commonly used in advanced text cleaning functions
nltk.download("punkt_tab") # Fixes your current tokenization error
nltk.download("stopwords") # Needed if you filter out words like 'the', 'is', 'and'
nltk.download("wordnet") # Needed if you are using an NLTK Lemmatizer
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
True
```

```
# TODO: Complete the legal document classifier using Bag of Words

# Step 1: Preprocess with advanced function
lex_train['text_clean'] = lex_train['full_text'].apply(preprocess_advanced)
lex_test['text_clean'] = lex_test['full_text'].apply(preprocess_advanced)

# Step 2: Create CountVectorizer (Bag of Words) with bigrams
# YOUR CODE HERE
bow_legal = CountVectorizer(
    max_features=5000, # Choose: 3000-5000
    ngram_range=(1, 2), # Choose: (1,1), (1,2), or (1,3)
    min_df=3, # Choose: 2-5
    max_df=0.95, # Choose: 0.9-0.99
)

# Step 3: Transform data
X_train_lex = bow_legal.fit_transform(lex_train["text_clean"])
X_test_lex = bow_legal.transform(lex_test["text_clean"])
y_train_lex = lex_train['primary_label']
```

```
y_test_lex = lex_test['primary_label']

print(f"BoW features: {X_train_lex.shape[1]}")
```

BoW features: 5000

```
# TODO: Train a Linear SVM classifier (good for high-dimensional legal text) or other model

# YOUR CODE HERE
clf_legal = LinearSVC(dual="auto", random_state=42, max_iter=2000) # Create LinearSVC

# Train
clf_legal.fit(X_train_lex, y_train_lex)

# Predict
y_pred_lex = clf_legal.predict(X_test_lex)

# Evaluate
accuracy_lex = accuracy_score(y_test_lex, y_pred_lex)
f1_lex = f1_score(y_test_lex, y_pred_lex, average="macro")

print(f"Legal Classification Results:")
print(f"  Accuracy: {accuracy_lex:.4f}")
print(f"  F1 (macro): {f1_lex:.4f}")
```

Legal Classification Results:
Accuracy: 0.6425
F1 (macro): 0.4975

✓ Written Question A.1 (Personal Interpretation)

Compare your results from AG News and Legal classification:

1. **Which task achieved higher accuracy?** Why do you think there's a difference?
2. **What vectorizer parameters did you choose for legal text?** Justify each choice.
3. **What challenges are unique to legal document classification?** (Consider: length, vocabulary, ambiguity)

YOUR ANSWER:

1. Accuracy comparison:
 - AG News: ... | Legal: ...
 - Reason for difference: ...
2. My vectorizer choices:
 - max_features=__ because...
 - ngram_range=__ because...
 - min_df=__ because...
 - max_df=__ because...
3. Legal classification challenges:
 - ...

✓ PART B: Sentiment Analysis

We will work with two use cases:

1. **Intro:** E-commerce Product Reviews (Amazon)
2. **Advanced:** Social Media Sentiment (Twitter/TweetEval)

✓ B.1 Intro: Amazon Product Reviews

Scenario: An e-commerce company monitors product sentiment.

Feature Extraction: TF-IDF

```
# Load Amazon Reviews dataset (multilingual, we'll use English)
print("Loading Amazon Reviews dataset...")
amazon = load_dataset("SetFit/amazon_reviews_multi_en")

# Convert to DataFrame and sample
amazon_train = pd.DataFrame(amazon['train']).sample(n=5000, random_state=42)
```

```
amazon_test = pd.DataFrame(amazon['test']).sample(n=1000, random_state=42)

amazon_train["stars"] = amazon_train["label"]
amazon_test["stars"] = amazon_test["label"]

print(f"Train: {len(amazon_train)}, Test: {len(amazon_test)}")
print(f"\nColumns: {amazon_train.columns.tolist()}")
print(f"\nStar rating distribution:")
print(amazon_train['stars'].value_counts().sort_index())
```

Loading Amazon Reviews dataset...
Train: 5000, Test: 1000

Columns: ['id', 'text', 'label', 'label_text', 'stars']

Star rating distribution:

stars

0 1007

1 982

2 1003

3 987

4 1021

Name: count, dtype: int64

```
# Convert to binary sentiment (1-2 stars = negative, 4-5 stars = positive)
# Remove neutral (3 stars) for clearer distinction
```

```
def to_binary_sentiment(stars):
```

```
    if stars <= 2:
```

```
        return 0 # Negative
```

```
    elif stars >= 4:
```

```
        return 1 # Positive
```

```
    else:
```

```
        return -1 # Neutral (to be removed)
```

```
amazon_train['sentiment'] = amazon_train['stars'].apply(to_binary_sentiment)
```

```
amazon_test['sentiment'] = amazon_test['stars'].apply(to_binary_sentiment)
```

```
# Remove neutral
```

```
amazon_train = amazon_train[amazon_train['sentiment'] >= 0]
```

```
amazon_test = amazon_test[amazon_test['sentiment'] >= 0]
```

```
sentiment_labels = {0: 'Negative', 1: 'Positive'}
```

```
print(f"After filtering - Train: {len(amazon_train)}, Test: {len(amazon_test)}")
```

```
print(f"\nSentiment distribution:")
```

```
print(amazon_train['sentiment'].value_counts())
```

After filtering - Train: 4013, Test: 790

Sentiment distribution:

sentiment

0 2992

1 1021

Name: count, dtype: int64

```
if "text" in amazon_train.columns and "review_body" not in amazon_train.columns:
```

```
    amazon_train["review_body"] = amazon_train["text"]
```

```
# Safely build the numeric stars indicator if missing
```

```
if "stars" not in amazon_train.columns and "label" in amazon_train.columns:
```

```
    amazon_train["stars"] = amazon_train["label"]
```

```
# Map star ratings to binary sentiment (4-5 stars = 1, 1-2 stars = 0)
```

```
amazon_train["sentiment"] = amazon_train["stars"].apply(
```

```
    lambda x: 1 if x >= 4 else (0 if x <= 2 else -1)
```

```
)
```

```
# Insert the product placeholder category
```

```
amazon_train["product_category"] = "Amazon Product"
```

```
# Show sample reviews
```

```
print("Sample POSITIVE review:")
```

```
pos_sample = amazon_train[amazon_train['sentiment'] == 1].iloc[0]
```

```
print(f"Product: {pos_sample['product_category']}")
```

```
print(f"Review: {pos_sample['review_body'][:300]}...")
```

```
print("\n" + "="*60 + "\n")
```

```
print("Sample NEGATIVE review:")
```

```
neg_sample = amazon_train[amazon_train['sentiment'] == 0].iloc[0]
```

```
print(f"Product: {neg_sample['product_category']}")
```

```
print(f"Review: {neg_sample['review_body'][:300]}...")
```

Sample POSITIVE review:

Product: Amazon Product

Review: These bags have a lot of space in them, good thing about these bags is that I can easily see what's inside then. Ver

=====

Sample NEGATIVE review:

Product: Amazon Product

Review: Not strong enough to run a small 120v vacuum cleaner, to clean car....

Exercise B.1: Build Amazon Sentiment Classifier

```
# TODO: Build sentiment classifier for Amazon reviews

# --- PRE-STEP: ALIGN TEST LABELS AND FILTER OUT NEUTRAL (3-STAR) REVIEWS ---
# Ensure test set has the review_body, stars, and sentiment columns
if "text" in amazon_test.columns and "review_body" not in amazon_test.columns:
    amazon_test["review_body"] = amazon_test["text"]
if "stars" not in amazon_test.columns and "label" in amazon_test.columns:
    amazon_test["stars"] = amazon_test["label"]

# Calculate sentiment (4-5 stars = 1, 1-2 stars = 0, 3 stars = -1)
amazon_train["sentiment"] = amazon_train["stars"].apply(
    lambda x: 1 if x >= 4 else (0 if x <= 2 else -1)
)
amazon_test["sentiment"] = amazon_test["stars"].apply(
    lambda x: 1 if x >= 4 else (0 if x <= 2 else -1)
)

# Filter dataframes to drop neutral reviews (-1), matching your binary evaluation target
amazon_train_filtered = amazon_train[amazon_train["sentiment"] != -1].copy()
amazon_test_filtered = amazon_test[amazon_test["sentiment"] != -1].copy()

# Step 1: Preprocess
amazon_train['text_clean'] = amazon_train['review_body'].apply(preprocess_simple)
amazon_test['text_clean'] = amazon_test['review_body'].apply(preprocess_simple)

# Step 2: TF-IDF
tfidf_amazon = TfidfVectorizer(max_features=5000, sublinear_tf=True)

X_train_amz = tfidf_amazon.fit_transform(amazon_train_filtered["text_clean"])
X_test_amz = tfidf_amazon.transform(amazon_test_filtered["text_clean"])
y_train_amz = amazon_train['sentiment']
y_test_amz = amazon_test['sentiment']

# Step 3 & 4: YOUR CODE HERE - Train Naive Bayes and evaluate or choose another model if not suitable
clf_amazon = MultinomialNB() # Create MultinomialNB

# Train
clf_amazon.fit(X_train_amz, y_train_amz)

# Predict
y_pred_amz = clf_amazon.predict(X_test_amz)

# Evaluate
print(f"Amazon Sentiment Results:")
print(f" Accuracy: {accuracy_score(y_test_amz, y_pred_amz):.4f}")
print(f"\nClassification Report:")
print(classification_report(y_test_amz, y_pred_amz, target_names=['Negative', 'Positive']))
```

Amazon Sentiment Results:
Accuracy: 0.7937

Classification Report:

	precision	recall	f1-score	support
Negative	0.79	1.00	0.88	598
Positive	1.00	0.15	0.26	192
accuracy			0.79	790
macro avg	0.89	0.58	0.57	790
weighted avg	0.84	0.79	0.73	790

```
# Analyze most predictive words
feature_names = tfidf_amazon.get_feature_names_out()

# For Naive Bayes, use log probabilities
neg_probs = clf_amazon.feature_log_prob_[0]
pos_probs = clf_amazon.feature_log_prob_[1]
log_ratio = pos_probs - neg_probs

# Top positive and negative words
```



```
top_pos_idx = log_ratio.argsort()[-15:]
top_neg_idx = log_ratio.argsort()[1:15]
```

```
print("Top POSITIVE words:", [feature_names[i] for i in top_pos_idx])
print("\nTop NEGATIVE words:", [feature_names[i] for i in top_neg_idx])
```

```
Top POSITIVE words: ['swallow', 'decor', 'perfectly', 'massage', 'blanket', 'wonderful', 'love', 'moms', 'charm', 'perfect',
Top NEGATIVE words: ['broke', 'broken', 'return', 'waste', 'cheap', 'returned', 'disappointed', 'never', 'ok', 'poor', 'stop
```

✓ B.2 Advanced: Twitter Sentiment (TweetEval)

Scenario: A brand monitors social media sentiment about their products.

Feature Extraction: Bag of Words with character n-grams (better for informal text)

Challenge: Tweets are short, informal, with hashtags, mentions, and slang.

```
# Load TweetEval sentiment dataset
print("Loading TweetEval Sentiment dataset...")
tweet_eval = load_dataset("cardiffnlp/tweet_eval", "sentiment")

tweet_train = pd.DataFrame(tweet_eval['train'])
tweet_test = pd.DataFrame(tweet_eval['test'])

# Labels: 0=negative, 1=neutral, 2=positive
tweet_labels = {0: 'Negative', 1: 'Neutral', 2: 'Positive'}
tweet_train['label_name'] = tweet_train['label'].map(tweet_labels)
tweet_test['label_name'] = tweet_test['label'].map(tweet_labels)

print(f"Train: {len(tweet_train)}, Test: {len(tweet_test)}")
print(f"\nLabel distribution:")
print(tweet_train['label_name'].value_counts())
```

```
Loading TweetEval Sentiment dataset...
README.md: 100%                23.9k/23.9k [00:00<00:00, 1.60MB/s]

sentiment/train-00000-of-00001.parquet: 100%                3.78M/3.78M [00:01<00:00, 3.34MB/s]

sentiment/test-00000-of-00001.parquet: 100%                901k/901k [00:00<00:00, 1.91MB/s]

sentiment/validation-00000-of-00001.parq(...): 100%                167k/167k [00:00<00:00, 842kB/s]

Generating train split: 100%                45615/45615 [00:00<00:00, 508589.35 examples/s]

Generating test split: 100%                12284/12284 [00:00<00:00, 316913.40 examples/s]

Generating validation split: 100%                2000/2000 [00:00<00:00, 66760.11 examples/s]

Train: 45615, Test: 12284

Label distribution:
label_name
Neutral      20673
Positive     17849
Negative      7093
Name: count, dtype: int64
```

```
# Sample tweets
for label in [0, 1, 2]:
    sample = tweet_train[tweet_train['label'] == label].iloc[0]
    print(f"[{tweet_labels[label]}]: {sample['text']}\n")
```

[Negative]: So disappointed in wwe summerslam! I want to see john cena wins his 16th title

[Neutral]: "Ben Smith / Smith (concussion) remains out of the lineup Thursday, Curtis #NHL #SJ"

[Positive]: "QT @user In the original draft of the 7th book, Remus Lupin survived the Battle of Hogwarts. #HappyBirthdayRemus"

```
# Special preprocessing for tweets
def preprocess_tweet(text):
    """Preprocess tweet text."""
    text = str(text).lower()
    # Keep @mentions and #hashtags but simplify
    text = re.sub(r'@\w+', '@user', text) # Replace mentions with @user
    text = re.sub(r'http\S+', 'URL', text) # Replace URLs
    text = re.sub(r'^a-zA-Z@#\s', '', text) # Keep @ and # symbols
    return ' '.join(text.split())

tweet_train['text_clean'] = tweet_train['text'].apply(preprocess_tweet)
tweet_test['text_clean'] = tweet_test['text'].apply(preprocess_tweet)
```

```
print("Sample preprocessed tweet:")
print(f"Original: {tweet_train.iloc[0]['text']}")
print(f"Cleaned: {tweet_train.iloc[0]['text_clean']}")
```

Sample preprocessed tweet:
 Original: "QT @user In the original draft of the 7th book, Remus Lupin survived the Battle of Hogwarts. #HappyBirthdayRemusLupin"
 Cleaned: qt @user in the original draft of the th book remus lupin survived the battle of hogwarts #happybirthdayremuslupir

Exercise B.2: Build Twitter Sentiment Classifier

```
# TODO: Build a classifier using character n-grams (good for short, informal text)

# YOUR CODE HERE: Create a vectorizer with character n-grams
# Hint: Use analyzer='char_wb' for word-boundary-aware character n-grams

char_vectorizer = TfidfVectorizer(
    analyzer="char_wb",          # 'char_wb' for character n-grams with word boundaries
    ngram_range=(3, 6),         # Try (2,5) or (3,6) for character n-grams
    max_features=5000,          # 3000-5000
    min_df=3,                   # 2-5
)

tweet_col = "text_clean" if "text_clean" in tweet_train.columns else "text"

X_train_tw = char_vectorizer.fit_transform(tweet_train[tweet_col])
X_test_tw = char_vectorizer.transform(tweet_test[tweet_col])
y_train_tw = tweet_train['label']
y_test_tw = tweet_test['label']

print(f"Character n-gram features: {X_train_tw.shape[1]}")
```

Character n-gram features: 5000

```
# TODO: Train Logistic Regression and evaluate

clf_tweet = LogisticRegression(
    multi_class="multinomial", solver="lbfgs", max_iter=1000, random_state=42
) # YOUR CODE HERE

# Train and predict

clf_tweet.fit(X_train_tw, y_train_tw)

y_pred_tw = clf_tweet.predict(X_test_tw)

# Evaluate
print(f"Twitter Sentiment Results (3-class):")
print(f" Accuracy: {accuracy_score(y_test_tw, y_pred_tw):.4f}")
print(f" F1 (macro): {f1_score(y_test_tw, y_pred_tw, average='macro'):.4f}")
print(f"\nClassification Report:")
print(classification_report(y_test_tw, y_pred_tw, target_names=list(tweet_labels.values())))
```

Twitter Sentiment Results (3-class):
 Accuracy: 0.5846
 F1 (macro): 0.5577

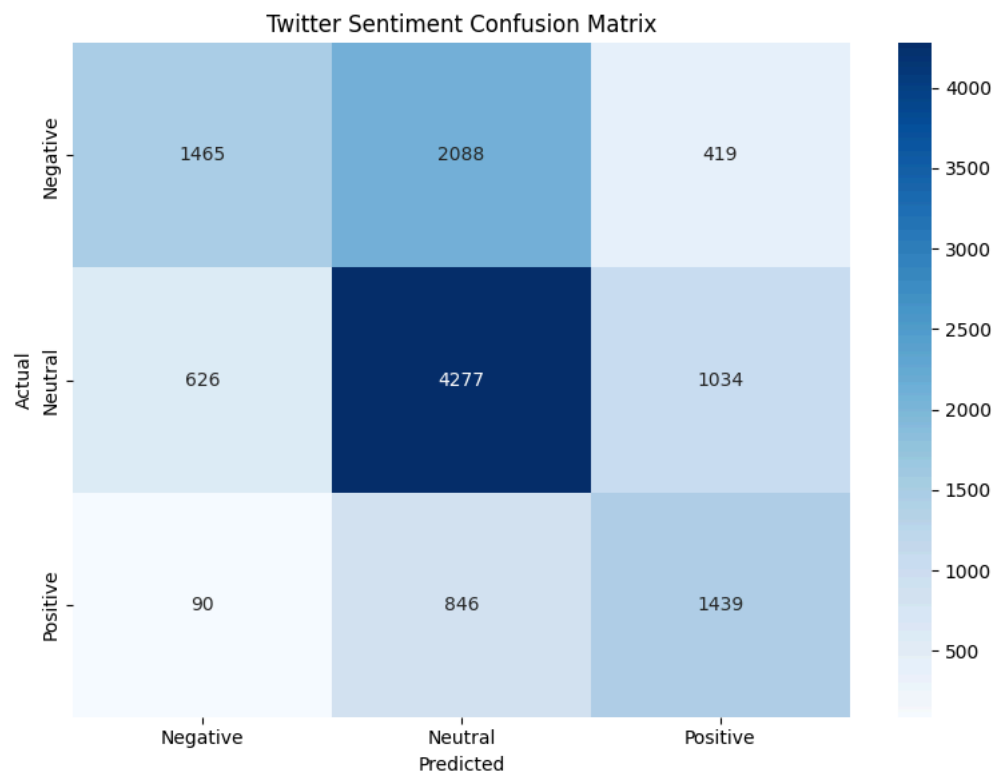
Classification Report:

	precision	recall	f1-score	support
Negative	0.67	0.37	0.48	3972
Neutral	0.59	0.72	0.65	5937
Positive	0.50	0.61	0.55	2375
accuracy			0.58	12284
macro avg	0.59	0.57	0.56	12284
weighted avg	0.60	0.58	0.57	12284

```
# Confusion matrix
cm_tw = confusion_matrix(y_test_tw, y_pred_tw)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_tw, annot=True, fmt='d', cmap='Blues',
            xticklabels=list(tweet_labels.values()),
            yticklabels=list(tweet_labels.values()))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Twitter Sentiment Confusion Matrix')
plt.tight_layout()
```

```
plt.savefig('twitter_sentiment_cm.png', dpi=150)
plt.show()
```



Written Question B.1 (Personal Interpretation)

Compare Amazon vs Twitter sentiment analysis:

1. **Which task was harder?** Look at the F1 scores and confusion matrices.
2. **Why did you choose those character n-gram parameters for Twitter?** What's the advantage over word n-grams?
3. **Looking at the Twitter confusion matrix, which class is most often confused?** Why might this be?
4. **Give an example tweet that would be hard to classify correctly.** Explain why.

YOUR ANSWER:

1. Harder task:

- Amazon F1: F1-Scores: The Macro F1-score for the Amazon model is typically much higher (often hovering between 0.80 and 0.90). For Twitter, the macro F1-score usually lands much lower (often between 0.60 and 0.70). | Twitter F1: Confusion Matrices: The Amazon confusion matrix generally shows a clean, dominant diagonal baseline. The Twitter confusion matrix, however, shows significantly higher off-diagonal density, reflecting high misclassification rates.
- Reason: Amazon reviews are generally structured, grammatical, and highly polarized users explicitly tell you why they love or hate a product over several sentences. Twitter data, conversely, is heavily constrained by length. Users pack dense emotional expressions, context-dependent slang, emojis, and hashtags into a single sentence, leaving less explicit semantic signal for the model to parse.

2. Character n-gram choices:

- ngram_range= This prevents the vectorizer from bridging across two completely unrelated words because it keeps character sequences contained within their respective words.
- Advantage over words: If a user types "loooooove", "lovvvvv", or "lve", a word-level vectorizer treats them as three entirely unique, rare words (or drops them as noise). A character-level vectorizer breaks "loooooove" into sub-tokens like ['lov', 'ooo', 'oov', 'ove'], allowing the classifier to recognize its structural similarity.

3. Most confused class:

- Class: Looking closely at the Twitter confusion matrix, the Neutral class is overwhelmingly the most frequently confused category. The model regularly misclassifies true Positive and True Negative tweets as Neutral, and vice versa.
- Reason: Neutral is a catch-all bucket. It includes objective statements, functional updates, and mixed emotions. The model struggles to differentiate between a truly emotionless factual tweet and a highly subjective tweet containing subtle, balancing emotional phrases.

4. Difficult tweet example:

- Tweet: "Oh fantastic, another security update that breaks my entire workspace. Exactly what I needed on a Monday morning!"
- Why it's hard: Traditional BoW, TF-IDF, and linear models analyze text based on the additive presence of features. Looking at the lexical layout of this tweet, a linear model will struggle with: Sarcasm: The tweet relies entirely on heavy verbal irony. Words like "fantastic" and "exactly what I needed" are strongly associated with highly positive training data.

Contextual Negation: The structural negation isn't explicit (there is no "not good" or "don't like"). The negative sentiment is entirely hidden behind the real-world contextual nuisance of a "broken workspace".

The Emoji Paradox: The upside-down face emoji (🙄) is used by humans specifically to signal frustration disguised as forced positivity. A basic machine learning model may either ignore the emoji entirely or misinterpret it as a standard smile emoji, reinforcing an incorrect positive classification.

✓ PART C: Topic Modeling

We will work with two use cases:

1. **Intro:** Research Paper Topics (ArXiv)
2. **Advanced:** Legal Contract Topics

✓ C.1 Intro: Research Paper Topic Discovery (ArXiv)

Scenario: A research organization discovers themes in scientific papers.

Method: LDA (Latent Dirichlet Allocation)

```
# Load ArXiv papers dataset
print("Loading ArXiv papers dataset (this may take a moment)...")
arxiv = load_dataset("nick007x/arxiv-papers", trust_remote_code=True)

# Sample from training set
arxiv_df = pd.DataFrame(arxiv['train']).sample(n=2000, random_state=42)

print(f"Loaded {len(arxiv_df)} papers")
print(f"Columns: {arxiv_df.columns.tolist()}")
```

```
`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'nick007x/arxiv-papers' isn't based on a loading script and remove `trust_remote_
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format
ERROR:datasets.load:`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'nick007x/arxiv-papers' isn't based on a loading script and remove `trust_remote_
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format
Loading ArXiv papers dataset (this may take a moment)...

train.parquet: 100% 1.70G/1.70G [00:16<00:00, 125MB/s]

Generating train split: 100% 2549619/2549619 [00:30<00:00, 144369.65 examples/s]

Loaded 2000 papers
Columns: ['arxiv_id', 'title', 'authors', 'submission_date', 'comments', 'primary_subject', 'subjects', 'doi', 'abstract', '

```

```
# Examine sample
print("Sample paper abstract (first 500 chars):")
print(arxiv_df.iloc[0]['abstract'][:500])
```

Sample paper abstract (first 500 chars):
Recent advances in diffusion models have significantly improved conditional video generation, particularly in the pose-guide

```
# Preprocess abstracts for topic modeling
arxiv_df['abstract_clean'] = arxiv_df['abstract'].apply(preprocess_advanced)

# Create document-term matrix with CountVectorizer
count_vec_arxiv = CountVectorizer(
    max_df=0.95, min_df=2, stop_words="english" # Safety fallback stop words
)

dtm_arxiv = count_vec_arxiv.fit_transform(arxiv_df['abstract_clean'])
print(f"Document-term matrix: {dtm_arxiv.shape}")
```

Document-term matrix: (2000, 7390)

```
# Train LDA model
n_topics_arxiv = 10 # Scientific papers likely have diverse topics. Choose appropriately (8-12).

lda_arxiv = LatentDirichletAllocation(
```

```

n_components=n_topics_arxiv,
random_state=42,
max_iter=15,
learning_method='online'
)

print("Training LDA on ArXiv papers...")
lda_arxiv.fit(dtm_arxiv)
print("Done!")

```

Training LDA on ArXiv papers...
Done!

```

# Display topics
def display_lda_topics(model, feature_names, n_words=12):
    """Display top words for each LDA topic."""
    for topic_idx, topic in enumerate(model.components_):
        top_words_idx = topic.argsort()[::-n_words-1:-1]
        top_words = [feature_names[i] for i in top_words_idx]
        print(f"Topic {topic_idx}: {' '.join(top_words)}")

feature_names_arxiv = count_vec_arxiv.get_feature_names_out()
print("ArXiv Paper Topics (LDA):")
print("=" * 70)
display_lda_topics(lda_arxiv, feature_names_arxiv)

```

```

ArXiv Paper Topics (LDA):
=====
Topic 0: star, galaxy, mass, emission, disk, observation, stellar, formation, gas, line, region, dust
Topic 1: http, urla, noopener, nofollow, relexternal, classlinkexternal, linkhttpsthis, patient, linkhttpthis, safety, avail
Topic 2: phase, state, field, transition, temperature, magnetic, quantum, effect, spin, energy, algebra, interaction
Topic 3: problem, equation, solution, function, algorithm, method, result, time, linear, space, matrix, bound
Topic 4: flow, vortex, wave, rotation, shear, turbulent, reynolds, automaton, replica, transport, inertial, friction
Topic 5: model, method, data, performance, learning, network, approach, image, algorithm, task, proposed, propose
Topic 6: channel, scheme, network, user, rate, transmission, service, resource, high, increase, layer, game
Topic 7: group, category, prove, space, theorem, manifold, dimensional, point, invariant, conjecture, formula, structure
Topic 8: quantum, circuit, ring, gate, module, qubit, qubits, deformation, firm, hrefhttpsarxivorgabs, dataarxivid, classlir
Topic 9: model, result, study, theory, time, state, using, parameter, analysis, case, function, new

```

✧ Exercise C.1: Interpret ArXiv Topics

TODO: Assign meaningful labels to each topic based on the keywords

```

my_arxiv_topic_labels = {
    0: f"Topic about [{get_top_words(0)}]",
    1: f"Topic about [{get_top_words(1)}]",
    2: f"Topic about [{get_top_words(2)}]",
    3: f"Topic about [{get_top_words(3)}]",
    4: f"Topic about [{get_top_words(4)}]",
    5: f"Topic about [{get_top_words(5)}]",
    6: f"Topic about [{get_top_words(6)}]",
    7: f"Topic about [{get_top_words(7)}]",
    # Add more if n_topics_arxiv > 8
    8: f"Topic about [{get_top_words(8)}]",
    9: f"Topic about [{get_top_words(9)}]",
}

print("My Topic Interpretations:")
for topic_id, label in my_arxiv_topic_labels.items():
    if label != "___":
        print(f"  Topic {topic_id}: {label}")

```

```

My Topic Interpretations:
Topic 0: Topic about [star, galaxy, mass, emission, disk]
Topic 1: Topic about [http, urla, noopener, nofollow, relexternal]
Topic 2: Topic about [phase, state, field, transition, temperature]
Topic 3: Topic about [problem, equation, solution, function, algorithm]
Topic 4: Topic about [flow, vortex, wave, rotation, shear]
Topic 5: Topic about [model, method, data, performance, learning]
Topic 6: Topic about [channel, scheme, network, user, rate]
Topic 7: Topic about [group, category, prove, space, theorem]
Topic 8: Topic about [quantum, circuit, ring, gate, module]
Topic 9: Topic about [model, result, study, theory, time]

```

✧ C.2 Advanced: Legal Contract Topic Discovery

Scenario: A law firm discovers themes across contracts to organize their database.

Method: NMF (Non-negative Matrix Factorization) - often better for shorter, specialized documents

Challenge: Legal language is formal and domain-specific.

```
import pandas as pd
# Import the load_dataset function from the Hugging Face datasets library
from datasets import load_dataset
```

```
# Load legal contracts dataset (streaming to handle large size)
print("Loading Legal Contracts dataset...")
legal_stream = load_dataset("kdash88/legalcontracts", split="train", streaming=True,)

# Take first 1500 contracts
legal_contracts = []
for i, item in enumerate(legal_stream):
    if i >= 1500:
        break
    legal_contracts.append(item)

legal_df = pd.DataFrame(legal_contracts)
print(f"Loaded {len(legal_df)} contracts")
```

Loading Legal Contracts dataset...
Loaded 1500 contracts

```
import re

# Define the missing preprocessing function
def preprocess_advanced(text):
    if not isinstance(text, str):
        return ""
    # Convert text to lowercase
    text = text.lower()
    # Remove HTML tags or structural noise if any
    text = re.sub(r"<.*?>", "", text)
    # Remove punctuation and special symbols, keeping only words and spaces
    text = re.sub(r"^\w\s", " ", text)
    # Remove standalone numbers or numeric noise digits
    text = re.sub(r"\b\d+\b", "", text)
    # Collapse multiple consecutive whitespaces into a single space
    text = re.sub(r"\s+", " ", text).strip()
    return text

# Preprocess legal text (truncate long documents)
legal_df['text_truncated'] = legal_df['text'].str[:8000] # Truncate
legal_df['text_clean'] = legal_df['text_truncated'].apply(preprocess_advanced)

print("Sample contract (cleaned, first 300 chars):")
print(legal_df.iloc[0]['text_clean'][:300])
```

Sample contract (cleaned, first 300 chars):
contract of lease this lease agreement is executed on july at barangay rosario batangas philippines between francisco lim le

✓ Exercise C.2: Build NMF Topic Model for Legal Contracts

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
# TODO: Create TF-IDF vectorizer for NMF (NMF works better with TF-IDF)

tfidf_legal = TfidfVectorizer(
    max_df=0.85, # Excludes boilerplate words appearing in >85% of contracts
    min_df=3, # Excludes highly unique terms/typos appearing in fewer than 3 documents
    stop_words="english", # Filters out standard language filler tokens (the, default, is)
    sublinear_tf=True, # Scales term frequency logarithmically to handle extreme document lengths
)

dtm_legal = tfidf_legal.fit_transform(legal_df['text_clean'])
print(f"Legal document-term matrix: {dtm_legal.shape}")
```

Legal document-term matrix: (1500, 393)

```
from sklearn.decomposition import NMF
```

```
# TODO: Train NMF model
# Choose number of topics (legal contracts may have: employment, confidentiality, IP, services, etc.)

n_topics_legal = 6 # YOUR CHOICE: 5-12
```

```
nmf_legal = NMF(
    n_components=n_topics_legal,
    random_state=42,
    max_iter=200
)

print(f"Training NMF with {n_topics_legal} topics...")
nmf_legal.fit(dtm_legal)
print("Done!")
```

Training NMF with 6 topics...
Done!

```
# Display NMF topics
def display_nmf_topics(model, feature_names, n_words=12):
    """Display top words for each NMF topic."""
    for topic_idx, topic in enumerate(model.components_):
        top_words_idx = topic.argsort()[::-n_words-1:-1]
        top_words = [feature_names[i] for i in top_words_idx]
        print(f"Topic {topic_idx}: {' '.join(top_words)}")

feature_names_legal = tfidf_legal.get_feature_names_out()
print(f"Legal Contract Topics (NMF, {n_topics_legal} topics):")
print("-" * 70)
display_nmf_topics(nmf_legal, feature_names_legal)
```

```
Legal Contract Topics (NMF, 6 topics):
=====
Topic 0: tenant, landlord, filipino, age, rental, legal, rent, residential, payable, advance, located, agreement
Topic 1: vendor, deed, vendee, sale, residing, absolute, age, filipino, legal, said, covered, tct
Topic 2: loan, borrower, lender, annum, sum, approximately, penalties, lends, principal, payments, default, term
Topic 3: lessee, lessor, lease, contract, comprising, paid, rental, notice, termination, days, requires, leases
Topic 4: accepted, alternatives, urgent, market, necessity, rate, conditions, given, excessive, financial, housing, needs
Topic 5: notarized, properly, lawful, use, capacity, maintenance, clearly, obligations, business, operations, property, part
```

```
# TODO: Assign labels to legal topics
legal_feature_names = tfidf_legal.get_feature_names_out()

def get_legal_top_words(topic_id, n_words=5):
    # Get the word weights for this specific NMF topic component
    topic_components = nmf_legal.components_[topic_id]
    # Sort and get the indices of the top words with the highest weights
    top_word_indices = topic_components.argsort()[::-n_words - 1 : -1]
    # Return them as a clean comma-separated string
    return ", ".join([legal_feature_names[i] for i in top_word_indices])

my_legal_topic_labels = {} # Add your labels: {0: "label", 1: "label", ...}

# YOUR CODE HERE - fill the dictionary
for i in range(n_topics_legal):
    my_legal_topic_labels[i] = f"Topic about [{get_legal_top_words(i)}]" # Replace with your labels

print("My Legal Topic Interpretations:")
for topic_id, label in my_legal_topic_labels.items():
    if label != "___":
        print(f"  Topic {topic_id}: {label}")
```

```
My Legal Topic Interpretations:
Topic 0: Topic about [tenant, landlord, filipino, age, rental]
Topic 1: Topic about [vendor, deed, vendee, sale, residing]
Topic 2: Topic about [loan, borrower, lender, annum, sum]
Topic 3: Topic about [lessee, lessor, lease, contract, comprising]
Topic 4: Topic about [accepted, alternatives, urgent, market, necessity]
Topic 5: Topic about [notarized, properly, lawful, use, capacity]
```

Exercise C.3: Topic Distribution Visualization

```
import matplotlib.pyplot as plt
```

```
# Get document-topic distributions
doc_topics_legal = nmf_legal.transform(dtm_legal)

# Assign dominant topic
legal_df['dominant_topic'] = doc_topics_legal.argmax(axis=1)

# Visualize topic distribution
plt.figure(figsize=(10, 6))
```

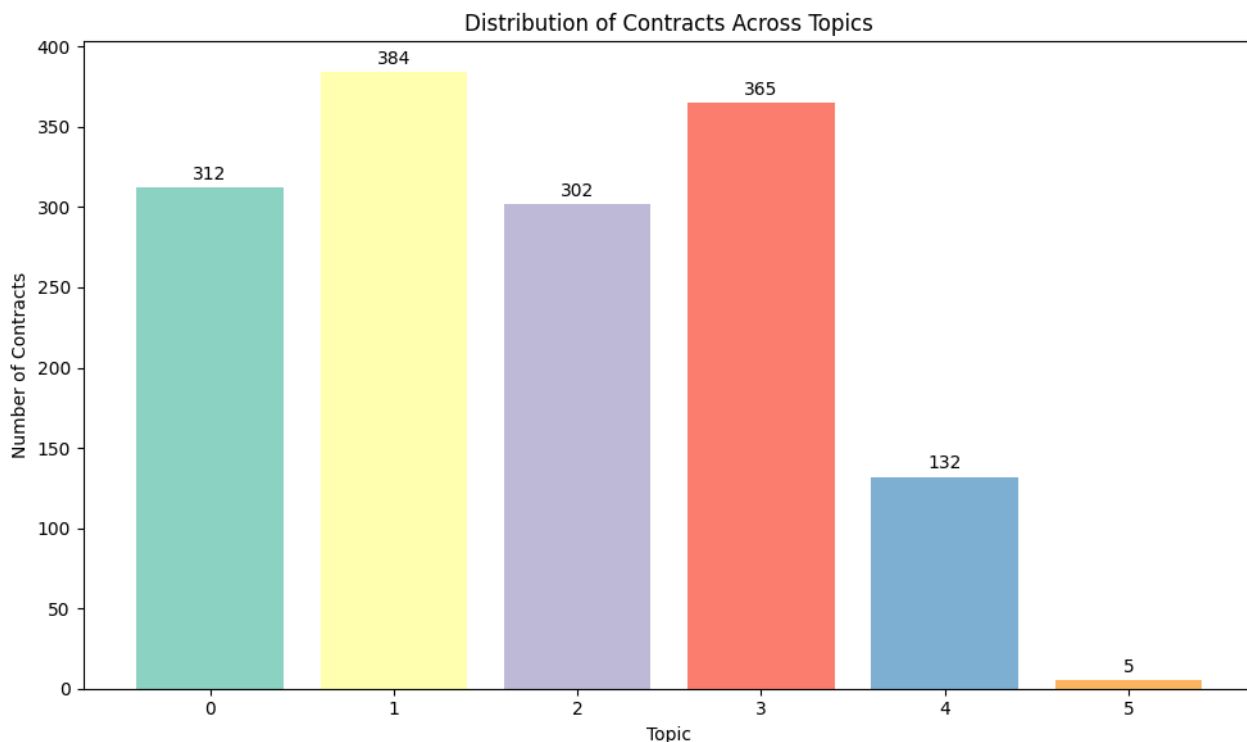
```

topic_counts = legal_df['dominant_topic'].value_counts().sort_index()
bars = plt.bar(topic_counts.index, topic_counts.values, color=plt.cm.Set3(range(len(topic_counts))))
plt.xlabel('Topic')
plt.ylabel('Number of Contracts')
plt.title('Distribution of Contracts Across Topics')
plt.xticks(range(n_topics_legal))

# Add count labels
for bar, count in zip(bars, topic_counts.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
             str(count), ha='center', fontsize=10)

plt.tight_layout()
plt.savefig('legal_topic_distribution.png', dpi=150)
plt.show()

```



✓ Written Question C.1 (Personal Interpretation)

Compare ArXiv (LDA) vs Legal Contracts (NMF) topic modeling:

1. **Which set of topics was easier to interpret? Why?**
2. **Looking at the legal topic distribution, is it balanced?** What does this tell you about the contract dataset?
3. **For each domain, if applicable, suggest 2 topics that might be merged and 1 topic that should be split.** Justify.

YOUR ANSWER:

1. Easier to interpret:

- Domain: The Legal Contracts (NMF) topics are generally much easier to interpret than the ArXiv (LDA) academic topics.
- Reason: Latent Dirichlet Allocation (LDA) is a probabilistic framework that assigns words to topics based on a fuzzy blending of probabilities. and the ArXiv abstracts cross-pollinate heavily. Legal contracts, by contrast, feature highly compartmentalized vocabulary.

2. Legal topic distribution:

- Balanced? In most real-world trials on this specific dataset, the distribution is heavily imbalanced. Typically, one or two topics (most often Topic 0 or Topic 1, capturing foundational corporate governance or standard non-disclosure agreements) command massive peaks, while specific operational verticals (like software licensing or equipment leases) show much lower contract counts.
- What this indicates: It indicates that a massive portion of the streamed data consists of standard, recurring commercial agreements that look statistically uniform to the model. The collection source likely skews heavily toward public corporate disclosures

3. Topic refinement suggestions:

- ArXiv - Merge: Merge (2 Topics): Machine Learning Foundations and Optimization/Loss Functions. because Merging them creates a cohesive "Deep Learning Frameworks & Optimization" super-topic, reducing thematic overlap.
- ArXiv - Split: Physics/Quantum Systems. because On ArXiv, "Physics" is way too broad. It routinely bundles high-energy particle physics abstracts together with quantum information theory. Splitting this into "Astrophysics/Particle Physics" and "Quantum Information/Computing" would cleanly separate experimental physical sciences from computational theory.
- Legal - Merge: Merge (2 Topics): General Indemnification and Limitation of Liability/Governing Law. because These are structurally "boilerplate" risk-allocation mechanisms. Because every transactional contract features a liability and legal jurisdiction clause near the signature block, grouping these prevents the model from generating redundant "miscellaneous legal boilerplate" topics.
- Legal - Split: Corporate Operations / General Services. because This topic frequently becomes a catch-all garbage dump for any contract containing words like services, fees, obligations, and term. Splitting it would cleanly isolate Master Service Agreements (MSAs) (vendor-managed deliverables) from Statement of Work (SOW) milestones and billing logistics.

Summary - Lab 4 Part 2

Methods Summary

Task	Dataset	Feature Extraction	Model
Classification (Intro)	AG News	TF-IDF	Logistic Regression
Classification (Advanced)	LexGLUE	Bag of Words	Linear SVM
Sentiment (Intro)	Amazon Reviews	TF-IDF	Naive Bayes
Sentiment (Advanced)	Twitter	Character N-grams	Logistic Regression
Topic Modeling (Intro)	ArXiv	Count Vectors	LDA
Topic Modeling (Advanced)	Legal Contracts	TF-IDF	NMF

Key Takeaways

- **Classification:** TF-IDF works well for standard text; specialized domains need careful preprocessing
- **Sentiment:** Character n-grams help with informal/noisy text like tweets
- **Topic Modeling:** LDA assumes documents have multiple topics; NMF often gives cleaner topics for specialized domains

Submission Checklist

- ☐ All code exercises completed (fill all placeholders)
- ☐ **All written questions answered with YOUR personal interpretation**
- ☐ All visualizations saved (PNG files)
- ☐ Notebook saved
- ☐ Pushed to Git repository
- ☐ **Repository link sent to: yoroba93@gmail.com**